

# mirror-pool

Behavioral anonymity and confidential value on Solana,  
verified on-chain

A Superteam Brasil “Privacy-Through-Noise” submission

Design paper, v1

## Abstract

On-chain behavior is legible. Even when a wallet reveals nothing directly, the *shape* of what it does (when it acts, how much it moves, who pays its gas, what software built its transaction, where its funds came from) is enough for commercial chain-analysis to cluster it, for copy-traders to shadow it, and for funding provenance to de-anonymize it. mirror-pool is an anonymity system over the *initiators* of an action rather than over funds. Its one-line thesis: **mirror-pool hides WHO initiated an action that everyone can see happened (two behavioral paths) AND, optionally, HOW MUCH moved (a confidential-value JoinSplit)**, a deliberate composition of two privacy axes, both proven on-chain on public Solana devnet. The behavioral core batches  $N$  identical actions into one synchronized epoch that settles atomically on a single block timestamp, paid by a rotating gasless relay, so the strongest empirical attack on mixer-style pools (FIFO temporal matching) has nothing left to order; an optional zero-knowledge opt-in path proves membership on-chain via Groth16 and settles to a fresh output with no participant signature, giving cryptographic who-initiated unlinkability. The optional confidential-value layer adds a 2-in/2-out Tornado-Nova JoinSplit that hides amounts inside encrypted notes. We report a falsifiable anonymity metric rather than a nominal count: an adversarial harness measures attacker advantage over the  $1/k$  guess, and an information-theoretic effective- $k$  model (Serjantov-Danezis  $2^{H(p)}$  plus min-entropy) shows that a naive pool advertising  $k = 32$  delivers an effective set of about 7.5 (worst case 1) under funding-provenance partitioning while mirror-pool holds the full 32. Everything is deployed and browser-verifiable on public devnet (program `EezWdFrmHtR2PCuucUruvkgYB9HW3w2KskZNeYmXszBq`), with two live soaks passing 17/17 behavioral and 25/25 confidential on-chain assertions, backed by 110 host tests and 42 on-chain program tests.

## 1 Introduction

A classic mixer breaks the link between a deposit and a withdrawal of *value*. mirror-pool breaks a different link: between an on-chain *action* and the *identity that initiated it*.  $N$  participants voluntarily pool the same action into one synchronized epoch; an observer sees  $N$  identical actions land together and cannot extract the per-initiator signal that chain-analysis tools depend on: timing, amount, gas payer, and wallet fingerprint. The anonymity set is the set of participants in the round, exactly as a mixer’s anonymity set is the set of deposits of one denomination. The funds are visible, the action is visible, the aggregate is visible; what is destroyed is which participant initiated which instance.

This is behavioral obscurity for the “privacy-through-noise” theme: the goal is to make on-chain *behavior* hard for automated chain-analysis to read. An optional confidential-value layer then goes past that theme and also hides *how much*: value lives in encrypted notes and moves through a Tornado-Nova-style zero-knowledge JoinSplit, so a private transfer reveals neither initiator nor amount. Composing the two axes is the point, not scope creep: a deployment that runs both hides who acted and how much moved at once.

**Contributions.**

- **Two peer behavioral paths** over one accumulator, one epoch clock, one  $k$ -floor, and one anti-Sybil economy: a synchronized-crowd path (behavioral intent-deniability against copy-trading and per-actor signal extraction) and a ZK-deniable opt-in path (cryptographic who-initiated unlinkability via an on-chain Groth16 membership proof).
- **An optional confidential-value layer:** a clean-room 2-in/2-out JoinSplit (**shield** / **transfer** / **unshield**) with value conservation, range proofs, `extDataHash` binding, and ECIES notes, composed with the behavioral guarantee to hide both axes.
- **A falsifiable anonymity story:** an adversarial A/B harness that *measures* FIFO/amount/fingerprint/gas-payer attacker advantage, plus an information-theoretic effective- $k$  model on the advertised-vs-effective axis the empirical mixer literature made memorable.
- **Trustless on-chain verification:** pure-Rust proving (no Node) and on-chain Groth16 verification via the BN254 pairing syscalls, so settlement checks a proof against a vendored verifying key rather than trusting a committee, all deployed and browser-verifiable on public devnet.

Every empirical claim in this paper is downstream of an executable artifact in the repository (a harness, a test, or a live soak), and Appendix A gives the exact command for each. Where the harness disagrees with the prose, the harness is the source of truth.

## 2 Problem and threat model

Threat models written for EVM chains under-count Solana’s observability, so we enumerate it first. Solana surfaces *more* structured per-transaction data than Ethereum, which makes several attacks that are approximate on EVM *exact* here.

| Observable                        | Why it matters                                                                                                                                                                                                             |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Balance deltas, first-class       | <code>pre/postBalances</code> and <code>pre/postTokenBalances</code> are transaction meta-data; exact per-account deltas are one RPC call away. Amount-matching is cheap and exact; a 0 pre-balance flags a fresh account. |
| Deterministic ATAs                | An Associated Token Account is a pure function of (owner, mint, program); any value landing in a canonical ATA re-links to its owner by table lookup.                                                                      |
| Signers and fee payer             | All signers are in the header; the fee payer is the first signer. Self-paid execution is self-attribution; co-signing merges clusters.                                                                                     |
| Compute-budget / fee fingerprints | <code>SetComputeUnitLimit/SetComputeUnitPrice</code> , tx version, instruction ordering, and ALT choice are visible and characteristic of wallet software.                                                                 |
| Timing despite Gulf Stream        | No lingering mempool, but Geyser/Yellowstone/shred observers see transactions pre-block (<100ms); post-block every tx has an exact slot and intra-slot position.                                                           |
| On-chain risk oracles             | Programs can gate on a wallet’s risk score at execution time, so a design that leaks the initiator leaks it to counterparty programs too.                                                                                  |

Table 1: What an observer actually sees on Solana. The one thing it lacks (a durable public mempool) buys a naive design nothing: the parseable intent is still there, observed a few milliseconds later.

**Adversaries.** We model four, all assumed to run full archival indexing and none assumed weak.

**A1 Chain-analysis clustering.** Account roll-up, ATA reversal, co-signer merging, and (the strongest anchor) funding-source anchoring: a fresh wallet funded from a KYC-clustered source is re-linked immediately.

**A2 Copy-traders / shadowers.** Parse swap intent (mint pair, amount) from pre-block validator streams and replay it within the slot. The lesson: *timing obfuscation alone loses*; the per-wallet parseable signal must be removed, not delayed.

**A3 The pool operator.** Infrastructure, not a trusted party; modeled honest-but-curious to adversarial. It can pad, censor, or reorder, but by construction it *cannot* substitute or retarget a committed action, nor make a participant act twice (epoch-scoped nullifier PDAs).

**A4 Sybil attacker.** Fills epochs with wallets it controls; if an epoch is 1 honest plus  $k-1$  attacker wallets, elimination de-anonymizes the honest one. This is how small mixers die, and it is why we never count nominal participants (Section 5).

**The real attacks (with the numbers that justify each defense).** We target the attacks the empirical literature identifies as the actual killers, not theoretical ones:

- **FIFO temporal matching:** linking each deposit to the earliest plausible later withdrawal linked up to **49%** of deposits on small Tornado pools and beat every other heuristic by 15 to 22 percentage points [1]. It is the single strongest known attack on mixer-style systems.
- **Amount matching:** variable and round-number amounts leak a large fraction of a claimed set. Wang et al. measured **27.34%** (Ethereum) and **46.02%** (BSC) anonymity-set reduction from composable heuristics [2].
- **Wallet fingerprinting:** matching gas/priority-fee settings across entry and exit cut Tornado’s effective set by **37%** on its own [1].
- **Self-paid gas and funding provenance:** not using a relayer, and funding from a clustered source, were both dominant de-anonymization vectors [1].

### 3 Design: behavioral anonymity

mirror-pool delivers two settlement paths that share one Poseidon accumulator, one slot-derived epoch clock, one on-chain  $k$ -floor, and one anti-Sybil economy. They are peers, not versions; a pool can run either or both.

#### 3.1 Protocol flow (commit, shared epoch, settle)

A participant computes a commitment  $C = \text{Poseidon}(\text{secret}, \text{actionHash}, \text{epoch})$  off-chain and posts only  $C$  on-chain during the round’s commit phase; the action and secret stay client-side. Every commit that lands in the same slot window belongs to one shared epoch, derived by all participants from the slot clock with zero coordination ( $\text{epoch} = \text{slot} / \text{epoch\_slots}$ ). Once the window closes at  $\text{settle\_slot} = (\text{epoch} + 1) \cdot \text{epoch\_slots}$ , the off-chain gasless coordinator checks the real  $k$ -floor and submits one settlement.

**Crowd path (Commit / SettleEpoch).**  $N$  participants each sign their own identical action. The coordinator composes them into a single versioned transaction that settles the whole epoch on one block timestamp:

ComputeBudget(limit) + ComputeBudget(price) + SettleEpoch{epoch, [nf...]} +  
participant\_0..participant\_ $k-1$  actions

Because every action in the epoch shares one timestamp, one fixed **ActionClass/SizeBucket** shape, and one relay-paid envelope, timing, amount, gas payer, and wallet fingerprint carry *no bits* about which participant is worth copying or attributing a strategy to. This is collective intent-deniability: the action still executes from the participant’s named wallet (so it stays attributable to that wallet), but it is indistinguishable *as a signal* from the rest of the crowd. **SettleEpoch** performs the anonymity bookkeeping (batching,  $k$ -floor, per-nullifier anti-replay, authority, fail-closed parsing); it does not cryptographically bind each nullifier to a distinct prior commitment, and the paper says so plainly. That binding is exactly what the ZK path adds.

**ZK opt-in path (CommitDeposit / SettleZk).** A participant escrows the action input at commit time, binding a *fresh* output through `actionHash = Poseidon(recipientHi128, recipientLo128, amount)`. At settlement the relay verifies a Groth16 membership proof on-chain that an output corresponds to *some* committed member without revealing which, checks the `actionHash` binding and nullifier, and releases the escrow to the fresh address with no participant signature. This provides cryptographic who-initiated unlinkability: the deposit is visible, the output goes to a fresh address, and the coordinator is not in the trust base for attribution (it settles for *some* member without learning which).

### 3.2 Why the design defeats the named attacks

| Attack                   | Mechanism that defeats it                                                                                                                                                                                                                                                                        |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FIFO temporal matching   | Shared-epoch batching: all actions settle on <i>one</i> block timestamp, so there is no per-participant settlement time to sort. Per-actor random delay does <i>not</i> achieve this (heavy-tailed timing survives); removing the ordering signal does. This mirrors Penumbrabatch clearing [5]. |
| Amount matching          | Fixed size buckets baked into the type system: <code>SizeBucket</code> is a field of <code>ActionClass</code> , one anonymity set per class, and the bucket is bound into the commitment, so a participant cannot commit one bucket and settle another.                                          |
| Wallet fingerprinting    | The coordinator submits every epoch with one pool-wide profile (identical CU limit, priority fee, tx version, account ordering, one shared ALT). There is no per-participant transaction to fingerprint.                                                                                         |
| Self-paid gas            | Settlement is gasless for participants; a <i>rotating</i> fee-payer set with independent funding histories signs and funds it, so no acting wallet self-attributes and no single relay becomes a consolidation cluster node.                                                                     |
| Copy-trading / shadowing | Before settlement the only thing on the wire is a commitment hash; at settlement <i>N</i> identical actions execute at once with no per-actor signal, so there is no individual to isolate and shadow.                                                                                           |

Table 2: Each defense maps to a concrete mechanism in the codebase, and each is measured (not asserted) by `crates/mirror-harness` (Section 5).

### 3.3 On-chain account and instruction model

Every account is a PDA whose address is a pure function of the pool and the epoch/nullifier/participant bytes; the program re-derives the expected address and rejects any mismatch (`InvalidPda`). Every layout is explicit byte offsets with bounds-checked little-endian accessors: no `unsafe`, no transmutes, no `repr(C)` casts on untrusted bytes. Parsing is fail-closed everywhere (wrong length, unknown tag, out-of-range count, or trailing bytes are rejected before any account is touched), and the wire layout is defined once and mirrored byte-for-byte between the host builder and the SBF parser with compile-time size asserts on both.

The Pool PDA (1780 bytes) holds one immutable config plus an inline depth-20 Poseidon frontier accumulator (up to about 1.05M leaves, so an append never touches a second account), a 32-root history ring (a membership proof is made against a root *snapshot*, so settle must accept any recent root), and the strictly-additive incentive counters (offsets 0...1762 are byte-identical to the pre-incentive layout). Anti-replay is one PDA per (pool, epoch, nullifier): its existence marks the nullifier spent, and scoping by epoch means the same secret yields a fresh, unlinkable nullifier in a later epoch. Immutability is a security property: a mutable *k*-floor would let an operator lower the floor right before a targeted epoch and shrink the set on demand.

**Solana specifics.** A settlement uses a v0 transaction with an Address Lookup Table to keep shared non-signer accounts (program ids, Pool PDA, clock, shared sink) out of the static list, while per-settlement accounts (Epoch PDA, Nullifier PDAs) and all signers stay static (Solana forbids loading a signer through an

| tag | instruction   | who submits    | effect                                                                                                                             |
|-----|---------------|----------------|------------------------------------------------------------------------------------------------------------------------------------|
| 0   | InitPool      | operator       | fix <code>epoch_slots</code> , <code>k_floor</code> , <code>entry_fee</code> , <code>reward_bps</code> , authority (all immutable) |
| 1   | Commit        | participant    | append one 32-byte leaf, bump commit count, collect fee, accrue dwell                                                              |
| 2   | SettleEpoch   | rotating relay | enforce authority + $k$ -floor, one Nullifier PDA per spend, mark settled                                                          |
| 3   | CommitDeposit | participant    | escrow <code>amount</code> , append leaf whose <code>actionHash</code> binds (recipient, amount)                                   |
| 4   | SettleZk      | rotating relay | verify Groth16 membership on-chain, release escrow to a fresh recipient                                                            |
| 5   | ClaimReward   | participant    | dwell-proportional, drain-safe reward-pool payout                                                                                  |
| 6   | InitValuePool | operator       | create the confidential ValuePool + vault; fix authority, fee, denomination                                                        |
| 7   | Transact      | rotating relay | verify one 2-in/2-out JoinSplit on-chain; shield / transfer / unshield                                                             |

Table 3: The instruction set. The first byte is the discriminator; the four keys of the account model are the Pool, Epoch, Nullifier, and Dwell PDAs.

ALT). The message must fit the 1232-byte packet limit, which caps the fixed-shape `PlainTransfer` baseline at 4 participants per settlement transaction (5 serialize to 1234 bytes); `plan_settlements` chunks a larger epoch into transaction-sized groups with disjoint nullifier subsets whose union is the whole epoch. Because Jito bundles cap at 5 transactions,  $N$ -party atomicity is program-side: `SettleEpoch` settles all intents in one instruction, so a no-show simply forfeits its slot and fee without stalling the epoch.

## 4 Confidential-value layer

Everything above hides who initiated while leaving every amount public. The confidential-value layer is the optional complement: a Tornado-Nova-style shielded pool that hides how much moves. It is a *separate* subsystem (its own account, accumulator, and two instructions), so a deployment can run the behavioral pool, the confidential pool, or both. When the two combine, a confidential transfer settles through the same gasless relay with no user signature and carries `publicAmount = 0`, so an observer learns neither which wallet initiated it nor how much it moved: the “who + how much” composition. The layer is a clean-room implementation of the public, open-source Tornado-Nova transaction circuit (the standard Poseidon JoinSplit), cited as prior art.

**The value-note (UTXO) scheme.** A value note is  $\{\text{amount}, \text{public\_key}, \text{blinding}\}$ , owned by a value keypair distinct from any Solana wallet:

$$\begin{aligned}
\text{public\_key} &= \text{Poseidon}(\text{private\_key}) \\
\text{commitment} &= \text{Poseidon}(\text{amount}, \text{public\_key}, \text{blinding}) \quad (\text{Merkle leaf}) \\
\text{signature} &= \text{Poseidon}(\text{private\_key}, \text{commitment}, \text{leafIndex}) \\
\text{nullifier} &= \text{Poseidon}(\text{commitment}, \text{leafIndex}, \text{signature})
\end{aligned}$$

The nullifier is bound to both the owner (through `signature`, which needs the private key) and the leaf position, so the same note at a different index yields a different nullifier and a double-spend collides. A dummy input has `amount = 0`; the circuit skips its Merkle-membership check, which is what lets a shield spend two dummy inputs.

**One universal statement.** A single 2-in/2-out JoinSplit (**Transact**, tag 7), distinguished only by the signed **publicAmount**, covers all three operations:

$$\text{shield} : \text{publicAmount} = v; \quad \text{unshield} : \text{publicAmount} = r - v; \quad \text{transfer} : \text{publicAmount} = 0.$$

The two ranges (shield in  $[0, 2^{248})$ , unshield in  $(r - 2^{248}, r)$ ) are disjoint, so the on-chain decoder recovers the sign unambiguously. **extDataHash** =  $\text{keccak256}(\text{recipient} \parallel \text{relayer} \parallel \text{fee\_be} \parallel \text{enc0} \parallel \text{enc1}) \bmod r$  is recomputed on-chain and required to match, so the relay cannot retarget the recipient, change the fee, or swap the encrypted payloads. The circuit (**circuits/transaction.circom**, depth 20, **27278 R1CS constraints**, 7 public inputs, 56 private inputs) enforces, per input, the key / commitment / signature / nullifier relations plus Merkle inclusion when amount  $\neq 0$ ; per output, the commitment and a 248-bit range bind; and globally, value conservation  $\sum \text{inAmount} + \text{publicAmount} = \sum \text{outAmount}$ , distinct input nullifiers, and tamper-evidence of **extDataHash**. The on-chain handler runs its checks in a fixed fail-closed order: authority, recent root, denomination (if pinned, checked before the expensive work), **extDataHash**, create each input nullifier PDA (an all-zero dummy sentinel is skipped), verify the proof, append both output commitments, move lamports per **publicAmount** keeping the vault rent-exempt, and emit the encrypted blobs as return data.

**Encrypted notes and fixed-denomination mode.** Each output commitment is opaque on-chain, so the sender posts an ECIES blob ( $\text{X25519 ECDH} \rightarrow \text{HKDF-SHA256} \rightarrow \text{ChaCha20-Poly1305}$ ; a fresh ephemeral key per message makes the AEAD (key, nonce) pair unique by construction). The plaintext is the 40 bytes the recipient does not already know ( $\text{amount}(8, \text{BE}) \parallel \text{blinding}(32)$ ), and the on-chain blob is a fixed 100 bytes ( $\text{ephemeral\_pub}(32) \parallel \text{nonce}(12) \parallel \text{ct} + \text{tag}(56)$ ). A recipient address splits a BN254 *value* key (spend authority, bound by the commitment) from an X25519 *viewing* key (encrypt and discover only), so a viewing key can go to an auditor without granting spend. The recipient trial-decrypts every blob (**scan**), keeps hits, and rebuilds each note to locate and later spend it. Setting a fixed **denomination** pins every public crossing to one magnitude (**DenominationMismatch** otherwise): the value analog of fixed size buckets, giving amount  $k$ -anonymity at the boundary.

This layer is an intentional composition, not scope creep. Its guarantee is stated precisely: it hides in-pool amounts and, for internal transfers and unshields, the initiator; it does *not* hide the public shield-in and unshield-out magnitudes or the pool TVL (a public on-chain balance), and its trusted setup is dev/test (Section 8).

## 5 Anonymity analysis

Overstating an anonymity set is the documented failure mode of prior mixers, so mirror-pool reports the *real* set. **KAnon{nominal, excluded}** with  $\text{real\_k} = \text{nominal} - \text{excluded}$  separates the nominal commit count from the real set (nominal minus operator-owned and detected Sybil decoys), and an epoch settles only when  $\text{real\_k} \geq \text{k\_floor}$ . Anonymity is  $1/\text{real\_k}$ , never  $1/\text{nominal}$ . We back this with two measurements.

### 5.1 A/B attack harness (advantage over the $1/k$ guess)

**crates/mirror-harness** implements four attacks over deterministic synthetic populations ( $n = 2048$  per scenario, fixed seed **0x4d4952524f52**, 50/50 train/test split) and reports attacker advantage  $\text{Adv} = P[\text{correct attribution}] - 1/\text{real\_k}$  for a *Baseline* (per-actor delay, self-paid gas, variable amounts, distinct funding roots) versus *MirrorPool* (shared-epoch batch, rotating gasless relay, fixed bucket). A positive measured advantage is a broken claim, full stop.

### 5.2 Effective- $k$ : advertised $k$ is not effective $k$

A pool that advertises  $k$  participants does not provide  $k$ -anonymity once an adversary partitions those participants by information it already holds. Following the standard information-theoretic metrics, we report the Shannon effective set  $2^{H(p)}$  (Serjantov-Danezis [3]) and the min-entropy worst-case set  $1/\max_i p_i$  (Diaz

| attack ( $k = 16$ , guess = 6.25%) | Baseline |          | MirrorPool |         |
|------------------------------------|----------|----------|------------|---------|
|                                    | acc      | adv      | acc        | adv     |
| FifoTemporalMatch                  | 83.79%   | +77.54pp | 5.96%      | -0.29pp |
| AmountMatch                        | 91.02%   | +84.77pp | 6.25%      | +0.00pp |
| WalletFingerprint                  | 88.77%   | +82.52pp | 6.25%      | +0.00pp |
| GasPayerReuse                      | 84.86%   | +78.61pp | 6.64%      | +0.39pp |

Table 4: Held-out attribution accuracy and advantage over random guessing at  $k = 16$  (full output at  $k = 4, 8, 16$  from `cargo run -p mirror-harness`). The headline: FIFO advantage collapses from +77.54pp under per-actor delay to -0.29pp under shared-epoch batching (statistically indistinguishable from 0), and the amount and fingerprint channels drop to exactly 0.00pp. Across  $k$ , FIFO advantage moves +72.46  $\rightarrow$  -2.73pp ( $k=4$ ), +79.20  $\rightarrow$  +0.20pp ( $k=8$ ), +77.54  $\rightarrow$  -0.29pp ( $k=16$ ).

et al. [4]) over the distribution  $p$  an adversary assigns after clustering committers. The dominant real leak modeled is **funding provenance**: a handful of exchange hot-wallets and faucets fund most participants (modeled as a Zipf common-funder distribution, about one funder per four committers), so the  $k$  committers collapse into a few equivalence classes and someone with a unique funder is alone in their class (worst case 1). On top of provenance we fold in the same timing, amount, and fingerprint channels the attack battery models.

| nominal $k$                                                          | Baseline $2^{H(p)}$ | Baseline worst-case | MirrorPool $2^{H(p)}$ |
|----------------------------------------------------------------------|---------------------|---------------------|-----------------------|
| <i>(a) funding-provenance channel alone (the marquee axis)</i>       |                     |                     |                       |
| 16                                                                   | 5.98                | 1                   | 16.00                 |
| 32                                                                   | 7.51                | 1                   | 32.00                 |
| 64                                                                   | 9.47                | 1                   | 64.00                 |
| <i>(b) all channels (provenance + timing + amount + fingerprint)</i> |                     |                     |                       |
| 16                                                                   | 1.01                | 1.00                | 16.00                 |
| 32                                                                   | 1.02                | 1.01                | 32.00                 |
| 64                                                                   | 1.01                | 1.00                | 64.00                 |

Table 5: A naive pool advertising  $k = 32$  delivers an effective set of about **7.5** (worst case **1**) under funding-provenance partitioning, and about **1.0** once every channel composes. mirror-pool holds the full nominal set at every  $k$ .

**Why mirror-pool holds effective- $k$  near nominal.** A behavioral channel informs the adversary only if settlement actually exposed it. mirror-pool settles one shared-epoch batch: every action carries one settle slot, one bucket amount, and one normalized fee shape, so the timing, amount, and fingerprint channels carry *zero variance* across the batch and contribute nothing. Funding via the shielded path breaks the provenance partition (one indistinguishable class). Both mechanisms together are why a naive pool’s effective set collapses while mirror-pool’s stays at nominal.

**Sybil dominance closes the “excluded = 0” gap.** When 75% of the nominal set is attacker-owned decoy traffic, an adversary that owns the decoys removes them, so effective- $k$  collapses to exactly  $\text{real\_}k = \text{nominal} - \text{excluded}$ : nominal 16/32/64 with 12/24/48 excluded gives effective- $k$  of 4.00/8.00/16.00. This makes KAnon’s honest subtraction an information-theoretic result rather than a bare assertion: an inflated nominal set buys no anonymity against anyone who can identify the decoys.

These effective- $k$  numbers are a *model* (deterministic synthetic populations, a modeled Zipf common-funder distribution), honestly labeled as such; an on-chain settlement-trace loader that computes effective- $k$  over real funding provenance is a named roadmap extension. The value is the structural claim on the same axis the empirical literature raised it.

**Anti-Sybil economy.** The metric is only as strong as its Sybil assumptions, so mirror-pool prices identity: each commit carries a fixed, non-refundable `entry_fee` (fidelity-bond style), collected on both the crowd `Commit` and the ZK `CommitDeposit`, so filling an epoch with  $k-1$  Sybils costs  $(k-1) \times \text{fee}$  per epoch forever (the floor rolls forward until it is met). A `reward_bps` share accrues to an on-chain reward pool that funds a *dwell* reward: `ClaimReward` pays a drain-safe share proportional to how many distinct epochs a participant has committed into, so staying (which thickens future epochs) is what pays, not merely showing up. Dwell advances only through a real, fee-paying commit, at most once per epoch, so it cannot be minted for free. The ZK path deliberately ships no identity-linked claim; its anonymity-preserving equivalent (a dwell/age ZK proof) is designed, not implemented, and the docs say so.

## 6 Implementation and on-chain verification

**Pure-Rust proving (no Node).** The participant CLI (`mirror-cli`) proves in-process in pure Rust by default: `ark-circom` loads the same compiled `circom .wasm` witness calculator and the same committed `.zkey` that `snarkjs` uses, computes the witness from typed Rust inputs, and `ark-groth16` over `ark-bn254` produces the proof using the `snarkjs`-compatible R1CS-to-QAP reduction, so the proof verifies under the same committed verifying key the on-chain program embeds. A legacy `--use-snarkjs` shell-out remains as an optional fallback. The CLI never self-submits: settlement is paid and signed by the rotating gasless relay, so the prove commands *emit* the settlement instruction bytes for the relay instead of sending them.

**Trustless on-chain Groth16 (a check, not a committee).** Both ZK paths verify their proof on-chain with `groth16-solana` (v0.2.0 byte layout) via the BN254 (`alt_bn128`) pairing syscalls, against a verifying key vendored into the program from `circuits/artifacts`. `proof_a` is emitted pre-negated so the on-chain path needs no ark serialization at runtime. This is the crucial contrast with a trusted-committee design: settlement does not trust an operator’s word that a member is valid; it checks a proof, and `verify()` (not `verify_unchecked()`) also rejects any public input not reduced modulo the field. The membership circuit (`circuits/membership.circom`, depth 20, **11522 R1CS constraints**, 4 public inputs, 41 private inputs) enforces leaf recomputation, Merkle inclusion, and the nullifier relation; public inputs are the fixed order `[root, nullifierHash, actionHash, epoch]`. Host and on-chain Poseidon (`light-poseidon` and `sol_poseidon`) are byte-identical and both match the circuit, pinned by a fixture cross-check test that reproduces the circuit’s `nullifierHash`, commitment leaf, and Merkle root exactly.

**Compute cost.** On-chain verification is cheap enough for per-membership settlement (about 170K to 500K CU for a 128-byte compressed proof); Section 7 gives the measured CU per flow on devnet.

**Coordinator.** `mirror-coordinator` maps its privacy jobs onto its modules: shared-epoch batching and the  $k$ -floor gate, honest `KAnon` accounting, the rotating gasless relay, and the normalized transaction profile (`cu_limit = 400,000`, priority fee = 10,000 micro-lamports). It is generic over a submitter seam, so the identical privacy-critical batching and floor logic runs against an in-memory submitter in tests and the real submitter in production, with no validator needed to test the decisions that matter.

## 7 Evaluation: public devnet deployment

Both soak suites were run against **public Solana devnet** (`api.devnet.solana.com`), a real, shared, public cluster. Every signature below is a real devnet transaction that resolves on the Solana Explorer; anyone can click through and verify it. This is devnet, not mainnet-beta: it proves the program deploys and every flow executes on a live public cluster exactly as it would on mainnet, without spending mainnet SOL. The SHA-256 of the on-chain program bytes equals the SHA-256 of the locally built `mirror_pool.so`, so the deployed program is exactly the source in this repository.

- **Program id:** EezWdFrmHtr2PCuucUruvkgYB9HW3w2KskZNeYmXszBq  
[explorer.solana.com/address/Eez...szBq](https://explorer.solana.com/address/Eez...szBq) (devnet)
- **Behavioral crowd settle** (4 identical actions, one atomic tx, one timestamp): [tx 4tcg...93NB](#)
- **Confidential private transfer** (publicAmount == 0, relay-only signer): [tx 2r6m...rtyv](#)

## 7.1 Behavioral soak: crowd + ZK settlement (17/17)

A fresh pool per run (epoch\_slots= 128, k\_floor= 3, entry\_fee=  $10^6$  lamports, reward\_bps= 2500). Four participants commit the *same* PlainTransfer into one shared epoch, settled by one atomic gasless transaction (the sink is credited  $4 \times 10^7$  lamports, i.e.  $4 \times$  the  $10^7$  bucket); a ZK opt-in escrow of  $5 \times 10^7$  is settled by an on-chain Groth16 SettleZk to a fresh recipient. The adversarial cases all fail closed on-chain: under-floor no-settle (BelowKFloor, custom error 0x2), duplicate nullifier (NullifierSpent, 0x3), re-settle (EpochAlreadySettled, 0x6), mismatched recipient (ActionHashMismatch, 0xe), and ZK replay (NullifierSpent). All 17/17 assertions passed.

| flow              | CU               | flow                | CU      |
|-------------------|------------------|---------------------|---------|
| init_pool         | 21,766           | underfloor_commit   | 39,499  |
| crowd_commit (x4) | 39,647 to 41,006 | crowd_settle        | 21,111  |
| zk_deposit_commit | 42,447           | zk_settle (Groth16) | 102,115 |

Table 6: Measured compute units per behavioral flow, finalized devnet transactions. The on-chain Groth16 membership verification (zk\_settle) runs in about 102K CU, comfortably inside a transaction’s budget alongside the settlement work.

## 7.2 Confidential-value soak: shield / transfer / unshield (25/25)

Fresh pools per run: a main ValuePool and a fixed-denomination ValuePool ( $10^7$  lamports). Amounts: shield  $5 \times 10^7$ , hidden transfer  $2 \times 10^7$ , withdraw  $2 \times 10^7$ ; the relay fee (5000 lamports) is bound into ext-data. The assertions prove the headline claim on-chain: the vault is credited by the shield deposit and the value root advances with both output commitments inserted; the private transfer carries publicAmount == 0 (verified as 32 zero bytes in the on-chain Transact data) and moves *no* public lamports, yet advances the root and spends its input notes; a recipient auto-discovers the payment note by scanning encrypted blobs; an unshield credits a fresh recipient and debits the vault by exactly the withdrawn amount; a wrong-denomination deposit is rejected on-chain (DenominationMismatch, 0x17) and refused client-side; a mutated public input is rejected (ProofVerificationFailed, 0xc); and conservation holds (vault = baseline + net deposit – net withdrawal). All 25/25 assertions passed.

| flow                  | CU      | flow                 | CU      |
|-----------------------|---------|----------------------|---------|
| init_value_pool_main  | 28,964  | transfer (JoinSplit) | 197,616 |
| init_value_pool_denom | 33,503  | unshield (JoinSplit) | 202,843 |
| shield (JoinSplit)    | 196,837 | denom_shield_exact   | 197,077 |

Table 7: Measured compute units per confidential flow, finalized devnet transactions. Each 2-in/2-out JoinSplit (7 public inputs) verifies on-chain in about 197K to 203K CU.

**Test suite.** Beyond the two live soaks, the repository ships **110 host tests** (cargo test --workspace, covering the Poseidon scheme cross-check against the circuit, the wire layout, KAnon accounting, the co-ordinator batching and  $k$ -floor logic, ECIES notes, and the effective- $k$  pure functions) and **42 on-chain program tests** (mollusk integration tests plus in-crate unit tests). cargo build-sbf is green and cargo check --workspace passes.

## 8 Honest limitations

A privacy tool that is vague about its non-goals is a trap. We list what still leaks and what we do not claim.

- **Dev/test trusted setup.** Both Groth16 circuits (membership and the JoinSplit) use a single-contributor, reproducible dev setup whose phase-2 entropy is public. The committed verifying keys verify and the program embeds them, but the layer **MUST NOT** secure real value until a real multi-party ceremony is run. This is a soundness caveat, disclosed here rather than buried.
- **Devnet, not mainnet.** The deployment proves execution on a live public cluster; it is not a mainnet-beta production deployment, and the system is not audited. Do not rely on it to protect real activity.
- **Anti-Sybil is economic, not a Sybil oracle.** Real  $k$ -anonymity assumes participants are economically distinct. The per-identity entry fee raises the price of flooding a round, but same-funding-source Sybils are not computable from the 32-byte commit stream and inflate `real_k`. The honest worst case: your anonymity is at least the number of participants you personally believe are independent, and at most `real_k`.
- **Effective- $k$  is a model.** The numbers come from deterministic synthetic populations and a modeled Zipf common-funder distribution, not a live pool. A live-provenance loader is a named roadmap item.
- **The crowd path is behavioral, not cryptographic.** On the crowd path the participant signs their own action, so it stays attributable to that wallet, and the coordinator learns the participant-to-intent mapping while composing settlement (it reveals nothing the signed on-chain action did not). Use the ZK opt-in path for cryptographic who-initiated unlinkability. This is a disclosed distinction, not a hidden one.
- **Per-epoch, not cross-epoch.** An adversary with a prior over many epochs (intersection attacks) can narrow the set across epochs; neither path makes a cross-epoch claim, and mitigations are roadmap items.
- **Public boundary and membership.** Membership is public (like Tornado deposits); the confidential layer’s shield-in and unshield-out magnitudes and the pool TVL stay public; a  $k$ -floor roll-forward is observable; and a participant who sweeps a fresh output into a clusterable wallet re-links themselves. Compliance and association-set tooling is future work.

## 9 Related work

mirror-pool is an independent clean-room implementation built from public techniques and papers. **Tornado Cash** established the commitment/nullifier/Merkle anonymity set for funds; mirror-pool reuses that accumulator shape but for *initiators* of behavior, and its behavioral core is the complement of a mixer (amounts public, initiator signal unattributable). **Tornado-Nova** contributed the 2-in/2-out Poseidon JoinSplit that the confidential-value layer implements clean-room. **Privacy Pools** added association-set membership to let honest users dissociate from illicit deposits; a mirror-pool association-set layer is named future work rather than a current claim. **Penumbra**’s batch swaps, where all swaps in a block clear at one price so per-user ordering and attribution are eliminated by construction, are the direct precedent for shared-epoch settlement: who-initiated indistinguishability achieved structurally rather than through added noise [5]. **Token-2022 confidential transfers** hide the transferred amount while leaving sender, receiver, and mint public, so the entity graph stays intact; they are the complement to the behavioral core (amounts hidden but behavior legible), and composing an amount-privacy primitive with a behavior-privacy one is exactly the “who + how much” point.

**An honest tradeoff: transparent setups.** mirror-pool uses Groth16, which needs a per-circuit trusted setup (currently dev/test). Approaches built on STARKs or other transparent, post-quantum setups (for example Bulletproofs-style range proofs or STARK-based systems) avoid a trusted setup entirely, at the cost of larger proofs or higher verifier work. That is a real tradeoff, not a strict win for either side: Groth16 buys small, constant-size proofs and cheap on-chain pairing verification (the  $\sim 100K$  to  $200K$  CU measured

in Section 7) in exchange for a ceremony; a transparent setup removes the ceremony risk in exchange for proof size or verification cost. mirror-pool’s choice is pragmatic for on-chain Solana verification today, and its trusted-setup limitation is disclosed rather than obscured.

## 10 Conclusion

mirror-pool composes two privacy axes that the literature usually treats separately: *who* initiated an action (behavioral anonymity, via synchronized-crowd batching and an on-chain ZK membership proof) and *how much* moved (a confidential-value JoinSplit). It refuses to overstate anonymity, reporting a falsifiable metric backed by an adversarial harness and an information-theoretic effective- $k$  model, and it verifies its zero-knowledge proofs trustlessly on-chain rather than trusting a committee. Every headline number in this paper is reproducible from the repository, and the whole system is deployed and browser-verifiable on public devnet. Appendix A makes each number checkable.

## A Reproduce every number

All commands run from the repository root. The host workspace is one Cargo workspace; the on-chain program is a separate SBF crate excluded from it.

**Build.**

```
cargo build --workspace
cargo build-sbf --manifest-path programs/mirror-pool/Cargo.toml
```

**Tests (110 host, 42 program).**

```
cargo test --workspace
cargo test --manifest-path programs/mirror-pool/Cargo.toml
```

**Adversarial harness (the attack table and the effective- $k$  table).**

```
cargo run -p mirror-harness --release
```

Prints the Baseline-vs-MirrorPool advantage table (Section 5, FIFO  $+77.54 \rightarrow -0.29$ pp at  $k=16$ ) and the effective- $k$  tables (funding-provenance alone:  $k=32 \rightarrow 7.51$  Baseline vs 32.00 MirrorPool; all channels; and the Sybil-dominance case collapsing to `real_k`). Deterministic (seed `0x4d4952524f52`), so any reviewer re-derives them bit-for-bit.

**Public devnet soaks (the 17/17 and 25/25 assertion tables and the CU numbers).**

```
PROG=EezWdFrmHtR2PCuucUruvkgyB9HW3w2KskZNeYmXszBq
cargo run -p mirror-soak --
--rpc-url https://api.devnet.solana.com --program-id $PROG
--epoch-slots 128 --k-floor 3 --zk-amount 50000000
cargo run -p mirror-soak --bin mirror-soak-value --
--rpc-url https://api.devnet.solana.com --program-id $PROG
--shield-amount 50000000 --transfer-amount 20000000 --denomination 10000000
```

The full reproduce recipe (fresh program keypair, single funded master payer, deploy, and re-run against a fresh devnet or a local mainnet-mirror validator) is in `docs/PROOF.md`, which also lists every finalized transaction signature.

**Browser verification (no build required).** Open the program address and the two representative transactions on the Solana Explorer:

- Program: <https://explorer.solana.com/address/EezWdFrmHtR2PCuucUruvkgYB9HW3w2KskZNeYmXszBq?cluster=devnet>
- Crowd settle: <https://explorer.solana.com/tx/4tcgNnhbZe7YKM26F6SnXW1WctASqVEqQ9W4v4NQ85fDfkYe3eq7YqCr4n7bEogm7U5By17Lkc5Tqa7jmZx693NB?cluster=devnet>
- Confidential transfer: <https://explorer.solana.com/tx/2r6mo2YrDtjJP8bnVqRJXaVDgVLkxaAmtKAHvgkwYvWb8rchvd63QinFqbmuj5Gk2Se8M9g9ajcCkC5R6QNprtyv?cluster=devnet>

**Where each claim lives.** The threat model and the per-attack numbers are in `docs/THREAT_MODEL.md`; the two settlement paths and the account model in `docs/ARCHITECTURE.md`; the effective- $k$  metric and its honest limits in `docs/EFFECTIVE_K.md`; the incentive economy in `docs/INCENTIVES.md`; and the live results, signatures, and CU per flow in `docs/PROOF.md`.

## References

- [1] *Deanonymizing Tornado Cash: an empirical analysis of mixed flows*. arXiv:2510.09433. FIFO temporal matching links up to 49% of deposits on small pools (+15 to 22pp over other heuristics); wallet fingerprinting via gas/priority-fee settings reduces the effective set by 37% alone; self-paid (non-relayer) gas is a dominant de-anonymization vector.
- [2] Y. Wang et al. *On How Zero-Knowledge Proof Blockchain Mixers Improve, and Worsen User Privacy*. arXiv:2201.09035. 27.34% (Ethereum) / 46.02% (BSC) anonymity-set reduction from composable heuristics; anonymity mining attracts privacy-indifferent users who degrade the set.
- [3] A. Serjantov and G. Danezis. *Towards an Information Theoretic Metric for Anonymity*. PET 2002. Defines the effective anonymity-set size as  $2^{H(p)}$ .
- [4] C. Diaz, S. Seys, J. Claessens, B. Preneel. *Towards Measuring Anonymity*. PET 2002. The companion entropy metric and the min-entropy / worst-case reading of anonymity as  $1/\max_i p_i$ .
- [5] Penumbra batch swaps. All swaps in a block execute at one clearing price, so per-user ordering and attribution are eliminated by construction and only the batch aggregate is revealed; the precedent for shared-epoch settlement.
- [6] Token-2022 Confidential Transfers (Solana Program Library). Hide the transferred amount while leaving sender, receiver, and mint public; the complement to mirror-pool’s behavioral core.